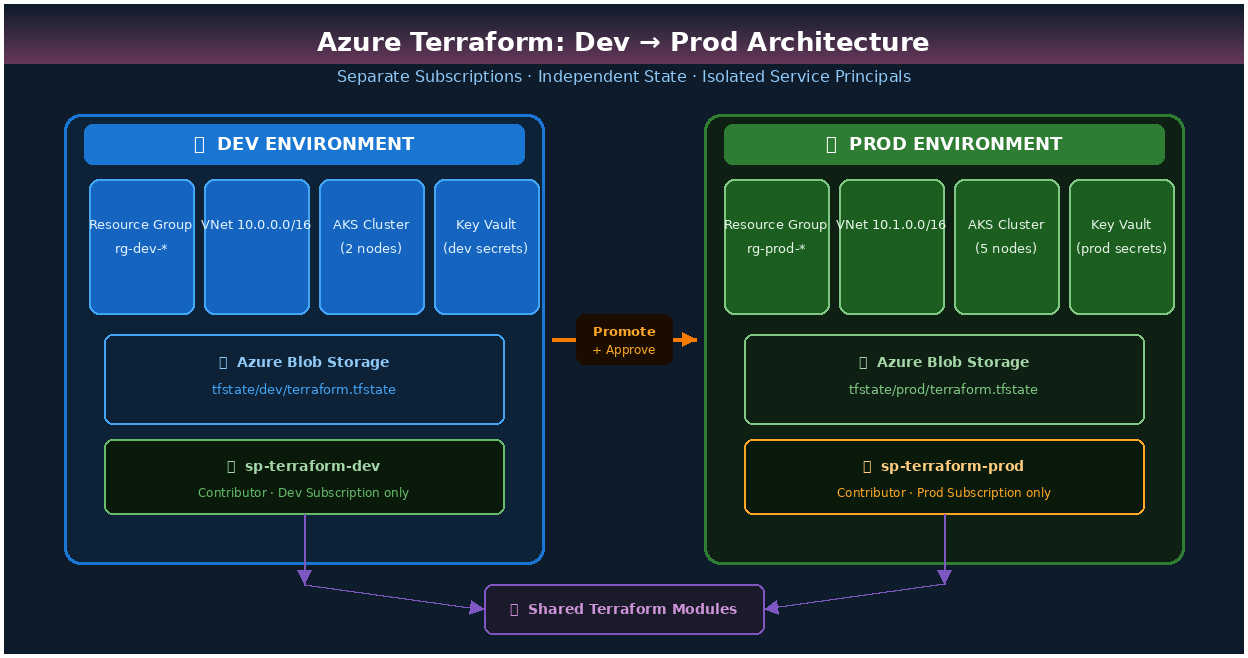
Terraform on Azure

Dev → Prod Environment

A Complete Practitioner Guide

Version 1.0 | Infrastructure as Code | Azure Cloud



High-level architecture showing Dev and Prod environment separation

1. Introduction

This document provides a comprehensive, step-by-step guide for using HashiCorp Terraform to manage and promote infrastructure from a Development (Dev) environment to a Production (Prod) environment on Microsoft Azure. Following the principles of Infrastructure as Code (IaC), every resource — virtual networks, Kubernetes clusters, databases, and secrets — is defined, versioned, and promoted through a controlled pipeline.

The guide covers folder structure conventions, workspace and backend isolation, variable management per environment, CI/CD pipeline design, remote state management, and production safety gates. It is intended for DevOps engineers, platform engineers, and cloud architects who want a repeatable, safe, and auditable deployment workflow.

1.1 Objectives

- Define and deploy Azure resources using reusable Terraform modules.
- Maintain complete environment isolation between Dev and Prod.
- Store Terraform state remotely in Azure Blob Storage with locking.
- Automate promotion via CI/CD with a mandatory manual approval gate.
- Enforce least-privilege access using separate Azure Service Principals.
- Ensure all changes are peer-reviewed before reaching production.

1.2 Prerequisites

Requirement	Details
Terraform	v1.5+ installed locally and in CI/CD agents
Azure CLI	az login configured; az account set to target subscription
Azure Subscription	Separate subscriptions recommended for Dev and Prod
Service Principal	One SP per environment with Contributor role
Azure Storage Account	Pre-created for remote state (one container: tfstate)
Git / Azure DevOps	Source control with branch protection on main
tfsec / tflint	Static analysis tools installed in CI agent

2. Project Folder Structure

Organising Terraform code into environment-specific directories with shared modules is the foundation of a maintainable multi-environment strategy. Each environment folder is self-contained and references shared modules by path or registry.

Recommended Terraform project folder layout for Azure multi-environment deployments

2.1 Directory Layout

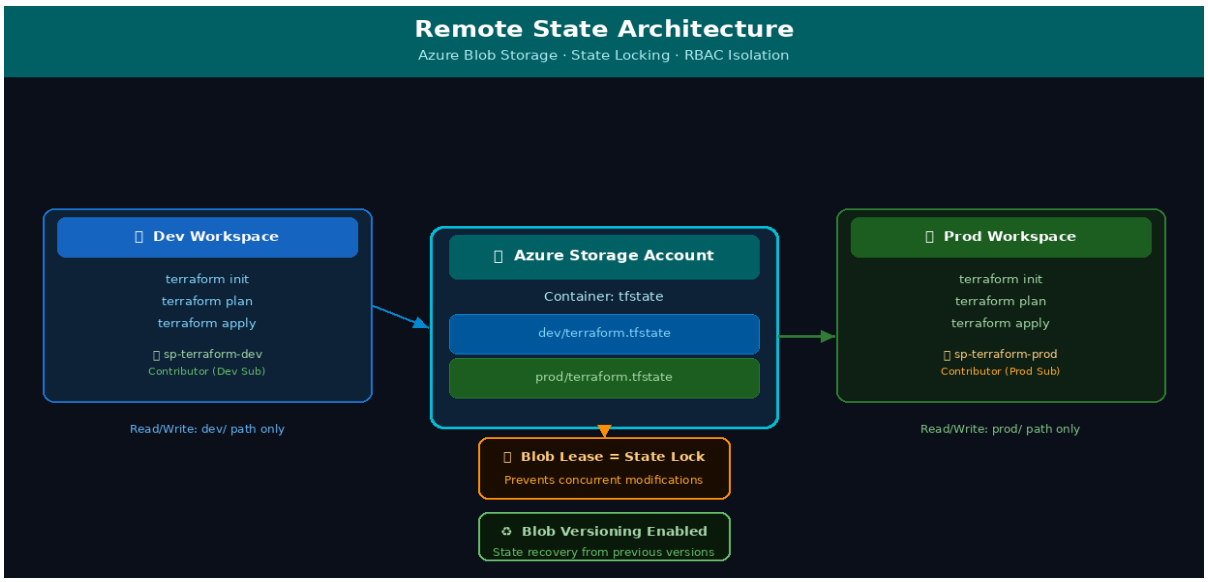
```
terraform-azure/
├── environments/
│   ├── dev/
│   │   ├── main.tf           # Root module — calls shared modules
│   │   ├── variables.tf      # Input variable declarations
│   │   ├── outputs.tf        # Exported values (IPs, IDs)
│   │   ├── terraform.tfvars  # Dev-specific values (overrides)
│   │   └── backend.tf        # Remote state: Azure Blob (dev/*)
│   └── prod/
│       ├── main.tf
│       ├── variables.tf
│       ├── outputs.tf
│       ├── terraform.tfvars  # Prod-specific values
│       └── backend.tf        # Remote state: Azure Blob (prod/*)
├── modules/
│   ├── networking/           # VNet, subnets, NSGs
│   ├── compute/              # AKS, VM Scale Sets
│   ├── database/             # Azure SQL, Cosmos DB
│   └── security/             # Key Vault, Managed Identity
└── README.md
```

2.2 Key Design Principles

- Each environment directory is independently initialised with its own backend.
- Modules under modules/ are the single source of truth — never duplicated.
- terraform.tfvars overrides differ per environment (SKU sizes, replica counts).
- No credentials or secrets appear in any .tf or .tfvars file.

3. Remote State Configuration

Terraform state tracks the real-world resources it manages. In a team environment, the state file must be stored remotely so that all team members and CI/CD agents operate on the same view of infrastructure. Azure Blob Storage provides reliable, HTTPS-accessible storage with native blob leasing used by Terraform for state locking.



Remote state isolation — Dev and Prod state files in separate blob paths with RBAC access control

3.1 Create the Storage Backend

Run the following Azure CLI commands once to set up the shared storage account and container:

```
# 1. Create resource group for Terraform state
az group create --name rg-terraform-state --location uksouth

# 2. Create storage account (globally unique name required)
az storage account create \
  --name tfstatemycompany \
  --resource-group rg-terraform-state \
  --sku Standard_LRS \
  --kind StorageV2 \
```

```
  --min-tls-version TLS1_2

# 3. Create the blob container
az storage container create \
  --name tfstate \
  --account-name tfstatemycompany

# 4. Enable versioning for state recovery
az storage account blob-service-properties update \
  --account-name tfstatemycompany \
  --enable-versioning true
```

3.2 Backend Configuration — Dev

Create environments/dev/backend.tf:

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstatemycompany"
    container_name       = "tfstate"
    key                  = "dev/terraform.tfstate"   # Unique key for dev
  }
}
```

3.3 Backend Configuration — Prod

Create environments/prod/backend.tf with a different key path:

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "tfstatemycompany"
    container_name       = "tfstate"
    key                  = "prod/terraform.tfstate"  # Unique key for prod
  }
}
```

4. Provider & Variable Configuration

4.1 Azure Provider Setup

In each environment's main.tf, configure the AzureRM provider. Credentials are injected via environment variables in CI/CD — never hardcoded:

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.90"
    }
  }
}


provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
  tenant_id       = var.tenant_id
  client_id       = var.client_id        # From ARM_CLIENT_ID env var
  client_secret   = var.client_secret    # From ARM_CLIENT_SECRET env var
}
```

4.2 Variables Declaration — variables.tf

```
# --- Identity ---
variable "subscription_id" { type = string }
variable "tenant_id"       { type = string }
variable "client_id"       { type = string }
variable "client_secret"   { type = string  sensitive = true }


# --- Environment ---
variable "environment"     { type = string  description = "dev | prod" }
variable "location"        { type = string  default     = "uksouth" }


# --- AKS ---
variable "aks_node_count"  { type = number }
variable "aks_vm_size"     { type = string }


# --- Database ---
variable "sql_sku_name"    { type = string }
variable "sql_storage_gb"  { type = number }
```

4.3 Variable Values — Dev vs Prod

environments/dev/terraform.tfvars:

```
environment    = "dev"
location       = "uksouth"
```

```
aks_node_count = 2
aks_vm_size    = "Standard_B2s"
sql_sku_name   = "Basic"
sql_storage_gb = 32
```

environments/prod/terraform.tfvars:

```
environment    = "prod"
location       = "uksouth"
aks_node_count = 5
aks_vm_size    = "Standard_D4s_v5"
sql_sku_name   = "GeneralPurpose"
sql_storage_gb = 512
```

5. Module Design

5.1 Networking Module

```
# modules/networking/main.tf
resource "azurerm_virtual_network" "main" {
  name                = "vnet-${var.environment}"
  address_space       = [var.address_space]
  location            = var.location
  resource_group_name = var.resource_group_name

  tags = {
    environment = var.environment
    managed_by  = "terraform"
  }
}

resource "azurerm_subnet" "app" {
  name                 = "snet-app-${var.environment}"
  resource_group_name  = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [var.app_subnet_cidr]
}
```

5.2 Compute Module (AKS)

```
# modules/compute/main.tf
resource "azurerm_kubernetes_cluster" "main" {
  name                = "aks-${var.environment}"
  location            = var.location
  resource_group_name = var.resource_group_name
  dns_prefix          = "aks-${var.environment}"

  default_node_pool {
    name       = "default"
    node_count = var.node_count
```

```
    vm_size    = var.vm_size
  }

  identity {
    type = "SystemAssigned"
  }

  tags = { environment = var.environment }
}
```

5.3 Calling Modules from Environments

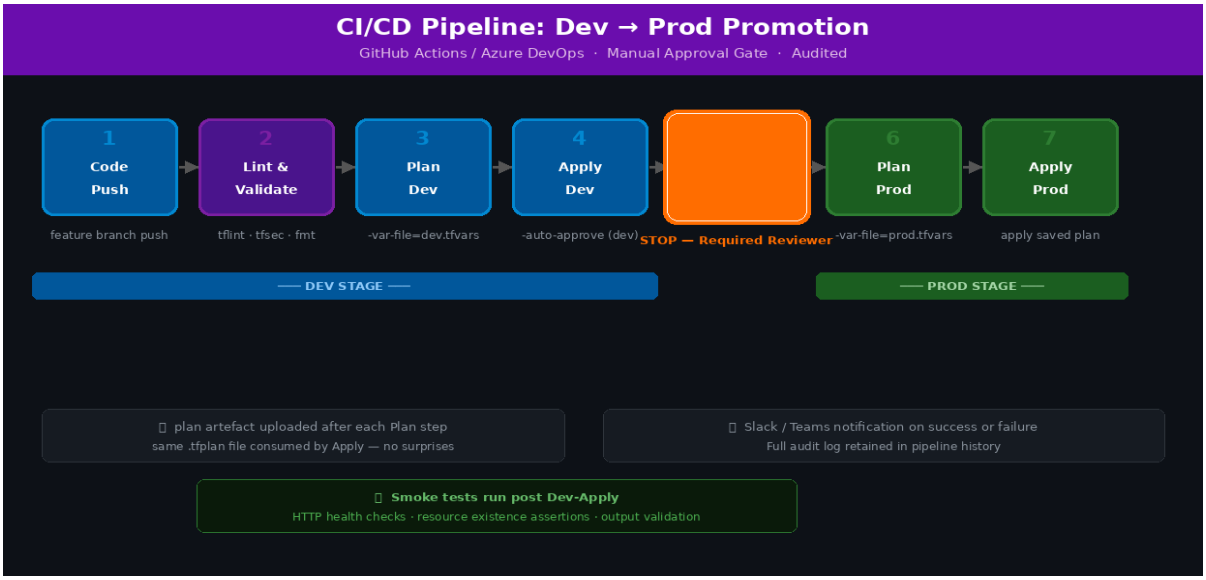
environments/dev/main.tf:

```
module "networking" {
  source              = "../../modules/networking"
  environment         = var.environment
  location            = var.location
  resource_group_name = azurerm_resource_group.main.name
  address_space       = "10.0.0.0/16"
  app_subnet_cidr     = "10.0.1.0/24"
}

module "compute" {
  source              = "../../modules/compute"
  environment         = var.environment
  location            = var.location
  resource_group_name = azurerm_resource_group.main.name
  node_count          = var.aks_node_count
  vm_size             = var.aks_vm_size
}
```

6. CI/CD Pipeline — Dev to Prod

Automating the promotion workflow removes human error from deployment steps while enforcing quality gates. The pipeline below is compatible with both Azure DevOps Pipelines and GitHub Actions.



Full CI/CD pipeline — code push through Dev apply, manual approval gate, and Prod deployment

6.1 Pipeline Stages

Stage	Description
1. Code Push	Developer pushes to feature branch; pull request triggers the pipeline
2. Lint & Validate	terraform validate, tflint for syntax; tfsec for security scan
3. Plan (Dev)	terraform plan -var-file=dev/terraform.tfvars — diff output stored as artefact
4. Apply (Dev)	Auto-approved apply to Dev environment; smoke tests run post-deploy
5. Approval Gate	Named reviewer must approve in Azure DevOps / GitHub Environments before Prod
6. Plan (Prod)	terraform plan -var-file=prod/terraform.tfvars — reviewed by approver
7. Apply (Prod)	terraform apply with explicit approval; notifications sent on success/failure

6.2 GitHub Actions Workflow

```
name: Terraform Dev → Prod
on:
  push:
    branches: [main]

env:
  ARM_CLIENT_ID:      ${{ secrets.ARM_CLIENT_ID_DEV }}
  ARM_CLIENT_SECRET: ${{ secrets.ARM_CLIENT_SECRET_DEV }}
  ARM_SUBSCRIPTION_ID: ${{ secrets.SUBSCRIPTION_ID_DEV }}
  ARM_TENANT_ID:      ${{ secrets.TENANT_ID }}
```

```yaml
jobs:
  deploy-dev:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.7.0" }

      - name: Terraform Init (Dev)
        run: terraform init
        working-directory: environments/dev

      - name: Terraform Plan (Dev)
        run: terraform plan -var-file=terraform.tfvars -out=plan.tfplan
        working-directory: environments/dev

      - name: Terraform Apply (Dev)
        run: terraform apply -auto-approve plan.tfplan
        working-directory: environments/dev

  deploy-prod:
    needs: deploy-dev
    runs-on: ubuntu-latest
    environment: production    # Requires manual approval
    env:
      ARM_CLIENT_ID:     ${{ secrets.ARM_CLIENT_ID_PROD }}
      ARM_CLIENT_SECRET: ${{ secrets.ARM_CLIENT_SECRET_PROD }}
      ARM_SUBSCRIPTION_ID: ${{ secrets.SUBSCRIPTION_ID_PROD }}
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Init (Prod)
        run: terraform init
        working-directory: environments/prod

      - name: Terraform Plan (Prod)
        run: terraform plan -var-file=terraform.tfvars -out=plan.tfplan
        working-directory: environments/prod

      - name: Terraform Apply (Prod)
        run: terraform apply -auto-approve plan.tfplan
        working-directory: environments/prod
```

7. Service Principals & Security

Each environment must have its own Azure Service Principal with the minimum permissions required. This limits the blast radius of a compromised credential and provides a clear audit trail.

7.1 Create Service Principals

```
# Dev Service Principal
az ad sp create-for-rbac \
  --name "sp-terraform-dev" \
  --role Contributor \
  --scopes /subscriptions/<DEV_SUBSCRIPTION_ID>

# Prod Service Principal
az ad sp create-for-rbac \
  --name "sp-terraform-prod" \
  --role Contributor \
  --scopes /subscriptions/<PROD_SUBSCRIPTION_ID>

# Grant state storage access (Blob Contributor — scoped to path)
az role assignment create \
  --assignee <SP_DEV_APP_ID> \
  --role "Storage Blob Data Contributor" \
  --scope "/subscriptions/<STATE_SUB>/resourceGroups/rg-terraform-state/...
```

7.2 Store Credentials in Key Vault

Store all Service Principal credentials in Azure Key Vault, not in pipeline variable groups where possible. Reference them at runtime using the az keyvault secret show command or the Key Vault Secrets action.

8. Promotion Commands — Step by Step

The following commands mirror what the CI/CD pipeline executes. They can also be run manually by an engineer for emergency changes or debugging.

8.1 Deploy to Dev

```
# Navigate to dev environment
cd environments/dev

# Initialise (downloads providers, configures remote backend)
terraform init

# Validate syntax and configuration
```

```
terraform validate

# Preview changes — always review this output
terraform plan -var-file=terraform.tfvars -out=dev.tfplan

# Apply changes
terraform apply dev.tfplan
```

8.2 Promote to Prod

```
# Switch to prod environment
cd ../prod

# Switch credentials to the Prod Service Principal
export ARM_CLIENT_ID="<PROD_CLIENT_ID>"
export ARM_CLIENT_SECRET="<PROD_CLIENT_SECRET>"
export ARM_SUBSCRIPTION_ID="<PROD_SUBSCRIPTION_ID>"

# Initialise prod backend
terraform init

# Plan against prod tfvars — REVIEW OUTPUT CAREFULLY
terraform plan -var-file=terraform.tfvars -out=prod.tfplan

# Apply only after explicit review and approval
terraform apply prod.tfplan
```

9. Troubleshooting

Issue	Resolution
State lock error: "state is already locked"	Another process holds the lock. Wait or force-unlock with terraform force-unlock <LOCK_ID> after confirming no concurrent run.
Authentication error: "AADSTS70011"	Service Principal secret has expired or is incorrect. Rotate the secret in Entra ID and update the CI/CD variable.
Error: "Resource already exists"	Resource exists but is not in state. Import it: terraform import azurerm_resource_group.main /subscriptions/.../resourceGroups/rg-dev.
"No changes" in prod but changes in dev	Ensure prod tfvars are correct. Check the provider version is the same in both environments.
Unexpected resource destroy in plan	Review changes carefully. May be caused by a renamed resource. Use moved {} blocks to rename without destroying.
Backend init fails with 403	The Service Principal lacks Storage Blob Data Contributor on the state container. Check RBAC assignment.

10. Quick Reference

Command	Purpose
terraform init	Initialise working directory; download providers; configure backend
terraform validate	Check configuration syntax and internal consistency
terraform fmt -recursive	Auto-format all .tf files to canonical style
terraform plan -out=plan.tfplan	Preview changes; save plan file for deterministic apply
terraform apply plan.tfplan	Apply the saved plan (no surprises)
terraform apply -auto-approve	Apply without confirmation prompt (CI/CD use only)
terraform destroy	Destroy all managed resources (use with extreme caution in prod)
terraform state list	List all resources tracked in the current state file
terraform state show <resource>	Show attributes of a specific state resource
terraform import <addr> <id>	Import an existing Azure resource into Terraform state
terraform force-unlock <id>	Release a stuck state lock (use only when safe)
terraform output	Display all output values from the current state

Always test changes in Dev before promoting to Prod.